KALASALINGAM ACADEMY OF RESEARCH AND EDUCATION

Presentation Content Builder AI

A Retrieval-Augmented Generation (RAG) System for Automated Presentation Content Creation

Project ID	PRJ017
Team ID	TO17
Team GitHub Repository	github.com/meghanaraghavaraju-collab/presentation-content-builder
Live Application	https://presentation-content-builder.vercel.app/

Student Details:

Student Name	**Registration Number**	**Individual GitHub Link**
A.Chandana	99230040861	https://github.com/Chandana2308
P.Meghana	99230040409	https://github.com/Meghana993-0PP
R.Meghana	99230040742	https://github.com/meghanaraghavaraju-collab
R.Divya Teja	99230040413	https://github.com/rdivyateja1293-netizen
Are navya showri Vasavika	99230040860	https://github.com/arenavyashowrivasavika-gif

Project Trainer / Guide: Mr.SHIRWIN PRINCE

Table of Contents:

Section	Content
1	Introduction
2	Problem Statement & Objectives
3	System Architecture (RAG Pipeline)
4	Implementation Details & Tech Stack
5	Results and Testing
6	Conclusion & Future Scope

1. Introduction

Creating an effective presentation from source material can require considerable time for reading, organizing information, drafting slide content, and preparing speaker notes. Presentation Content Builder AI is a full-stack application that uses Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) to transform uploaded documents into structured, presentation-ready slide decks. It supports multiple document formats, semantic retrieval, content synthesis, interactive editing, and export to a widescreen PowerPoint file. The generated slide content includes source references to support factual grounding.

2. Problem Statement & Objectives

Problem Statement: Users often need to manually extract key ideas from lengthy documents and convert them into clear, logically ordered slides. This process is time-consuming and may lead to missing context, inconsistent structure, or unsupported statements. Basic keyword search does not reliably retrieve semantically relevant material.

Objectives:

- Develop an AI-assisted system that converts uploaded source documents into structured slide-deck content.
- Support ingestion of PDF, DOCX, Markdown, TXT, CSV, and JSON files.
- Retrieve relevant content through semantic vector search and use it to ground generated slide material.
- Generate slide titles, core takeaways, bullet points, layout suggestions, speaker notes, and retrieved-sourcereferences.
- Provide an interactive slide editor and downloadable 16:9 PowerPoint (.pptx) output.
- Offer flexible model integration with OpenAI, Gemini, and a built-in heuristic RAG synthesis engine.

3. System Architecture (RAG Pipeline)

The system follows a document-ingestion stage and a query/retrieval-and-generation stage. Uploaded files are parsed and chunked with metadata and overlap; embeddings are stored in a vector store. The user's presentation request is then used to retrieve relevant chunks, which are supplied to the synthesis engine to produce output conforming to a strict SlideDeck JSON structure. The content is displayed in the web slide studio and can be exported as PPTX.

3. System Architecture (RAG Pipeline) — Details

3.1 Data Ingestion Phase

The application accepts multi-format source documents including PDF, DOCX, Markdown, text, CSV, and JSON. The ingestion pipeline extracts text, tracks document metadata, and divides content into manageable chunks using a Recursive Character Text Splitter. Chunk overlap helps preserve continuity between adjacent sections. The text chunks are converted into vector representations for semantic retrieval.

3.2 Retrieval, Query & Generation Phase

When the user requests presentation content, the system performs multi-aspect semantic retrieval, including relevant context, problem, pillars, and metrics. Retrieved chunks are passed to the LLM orchestration/synthesis engine. The engine generates structured slide content under a strict SlideDeck JSON schema. Retrieved source references are included to support factual grounding.

3.3 Output and Presentation Studio

- Interactive 16:9 slide canvas with in-place live editing.

- Speaker delivery notes pane, with notes embedded in exported presentation.
- Slide layout suggestions and dynamic card/shape layouts.
- Programmatic PowerPoint export to downloadable .pptx format.
- Factual grounding citations/source references associated with slide content.

4. Implementation Details & Tech Stack

Component	Technology / Approach	Purpose
Backend/API	Python; FastAPI REST endpoints	Core processing and application API.
RAG orchestration	LLM orchestration and retrieval pipeline	Connect ingestion, retrieval, and content synthesis.
Document processing	Multi-format loaders; Recursive Character Text Splitter	Extract and chunk source documents with metadata/overlap.
Vector storage	ChromaDB; serverless in-memory vector store	Store and retrieve document embeddings.
LLM synthesis	OpenAI / Gemini / built-in heuristic RAG engine	Generate presentation content grounded in retrieved context.
PowerPoint generation	PPTX export engine	Create downloadable 16:9 PowerPoint decks with presenter notes.
Deployment	Vercel serverless functions; local Python server	Deploy online or run locally.
Frontend	Interactive web Slide Deck Studio	Preview and edit generated slides in browser.

4.1 Deployment and Configuration

The repository provides a Vercel serverless entry point at api/index.py and routing configuration through vercel.json. Optional environment variables include OPENAI_API_KEY and GEMINI_API_KEY. The README describes the built-in heuristic RAG synthesis engine as available even when these API keys are not configured. For local development, dependencies are installed from requirements.txt and the application is launched using python run.py; API documentation is available at /docs.

5. Results and Testing

The project repository documents an automated test suite of 11 unit and integration tests. The tests cover the principal workflow and system components listed below. These are repository-reported test coverage areas; no additional accuracy, latency, or user-study measurements are asserted in this documentation.

- Multi-format document ingestion (PDF, DOCX, TXT, CSV, JSON).
- Recursive character text chunking.
- ChromaDB and serverless in-memory vector-store behavior.
- SlideDeck JSON schema validation.
- Generation of 16:9 widescreen PowerPoint (.pptx) presentations.

• FastAPI REST endpoints and end-to-end pipeline checks.

5.1 Expected System Output

For a successful presentation-generation request, the application produces a deck title and target audience, followed by slides containing a slide number, title, core takeaway, bullet points, visual layout suggestion, speaker notes, and retrieved sources. The web studio supports editing and previewing the deck, and the export engine generates the PowerPoint file.

6. Conclusion & Future Scope

Conclusion: Presentation Content Builder AI applies RAG to streamline the transformation of source documents into presentation-ready content. Its multi-format ingestion, semantic retrieval, LLM synthesis, structured slide schema, interactive editing studio, and PPTX export provide an integrated workflow for building presentations while maintaining links to retrieved source material.

Future Scope: Potential extensions include richer parsing of images, charts, and tables within source documents; enhanced citation verification; user authentication and role-based document access; collaborative slide editing; customizable themes and templates; feedback-based evaluation of retrieval and generation quality; and broader analytics for presentation quality.